

Technical Project Report

Project Overview

PixelCatsFork is a multi-project .NET solution built around small pixel-based games, shared rendering abstractions, and leaderboard integration. The codebase centers on the `ConsoleTest` project, which hosts the main game loop and game selection state machine, while `PixelBoardDisplay` provides a reusable display and input layer for both console emulation and Arduino-style hardware output. The `PixelCatsClient` project contains HTTP client utilities for leaderboard operations, and `ConsoleTest.Tests` provides unit tests for the game and client behavior.

The repository does not contain an internal web backend. Instead, it relies on an external leaderboard API and a shared file, `shared/latest_score.json`, to move score data between the console game loop and downstream consumers.

Problem Being Solved

The system addresses a practical game-utility problem: it needs to run lightweight arcade-style games, render their state on a small pixel board, export scores in a reliable format, and submit those scores to a leaderboard service. The design also needs to support both an emulator-first development workflow and a hardware-oriented deployment path.

Major Features

- Multiple games implemented through a shared interface in `ConsoleTest/Games/IGame.cs`.
- A console-hosted state machine in `ConsoleTest/Program.cs` for title, playing, and game-over transitions.
- Shared display abstractions in `PixelBoardDisplay/` with both `ConsoleDisplay` and `ArduinoDisplay` implementations.
- Atomic score export to `shared/latest_score.json`.
- HTTP client code in `ConsoleTest/Leaderboardclient.cs` and `PixelCatsClient/` for leaderboard and code-generation workflows.
- Automated unit tests in `ConsoleTest.Tests/`.
- MonoGame-based desktop projects in `Client/` and `HerdingCats/`.

Technical Challenges

- The game loop has to coordinate rendering, input, score export, and leaderboard submission without a web server or database to buffer those responsibilities.
- The display layer must support both console output and serial-port hardware output while sharing the same game pixel buffer.
- The score export pipeline has to remain safe under file-watcher style consumption, which is why `ConsoleTest/Program.cs` writes the JSON file atomically.
- The HTTP clients must tolerate inconsistent server payloads and failures because the server code is not part of the repository.

- The codebase mixes cross-platform .NET components with Windows-specific hardware hooks in `PixelBoardDisplay/ArduinoInput.cs` and `PixelBoardDisplay/ConsoleDisplay.cs`.

Implementation Approach

The implementation uses small, focused modules with clear responsibilities:

- `ConsoleTest/Program.cs` owns the application state machine, configuration loading, score export, and leaderboard submission.
- `ConsoleTest/Games/Snake.cs`, `Tetris.cs`, and `Education.cs` implement the game-specific update and draw logic behind `IGame`.
- `PixelBoardDisplay/DisplayHelper.cs` centralizes pixel board state and LCD string handling for both display backends.
- `PixelBoardDisplay/ConsoleDisplay.cs` renders to the local terminal emulator.
- `PixelBoardDisplay/ArduinoDisplay.cs` serializes frames and LCD updates for hardware.
- `PixelCatsClient/ApiClient.cs`, `PixelCatsClient/PixelCatsApiClient.cs`, and `ConsoleTest/Leaderboardclient.cs` encapsulate HTTP interactions.
- `ConsoleTest.Tests/` verifies the client and game behaviors without requiring real hardware or a live API.

Technical Achievement

The strongest engineering feature of the repository is the separation between game logic, rendering, persistence, and HTTP transport. This makes the project easier to test and easier to reason about than a single monolithic game script. The solution also demonstrates:

- interface-driven design with `IGame` and `IDisplay`;
- emulator and hardware support through interchangeable display implementations;
- atomic file writing for score handoff;
- defensive API client code that treats failure as a normal case;
- test coverage that validates the behavior of the most important integration points.

Limitations

- No internal backend service is included in the repository, so the API server is an external dependency.
- No internal database schema or migration history is present.
- Hardware paths assume a Windows environment and a serial device configuration that is hard-coded in `SerialPortManager.cs`.
- Some client code contains fallback parsing logic because the server response shape is not fully controlled locally.

Future Improvements

- Extract shared data transfer objects so the API payload contract is defined in one place.
- Replace hard-coded hardware defaults with configuration-driven serial settings.